Replication of Unity's Physics Engine

By: Isaac Brinkman

Advisor: Dr. Choong-Soo Lee

Goal

eplication of Unity's physics engine in order to gain a erstanding for the underlying code hidden from the me developer. As well as customize the engine in order easier to do certain processes.

Introduction

ysics engine allows for thousands of objects to be a single frame (less than a second). This handling dding forces (friction, gravity, etc.), collision ons, and other calculations. Many of these processes thout the developer knowing what exactly is going on fects it will have on the game they are creating. In this, I gained insight in how the Unity Engine handles hysics operations.

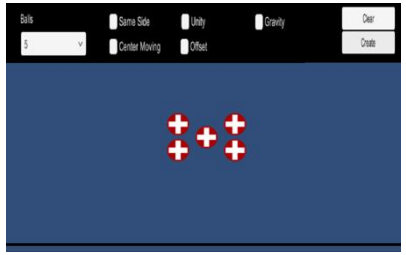
Differences

Unity:	My Physics:
ness and friction	• Set Bounciness of 1 and friction of 0
ons between complex	• Collision between squares and circles
n on collision	• No Rotation on collision

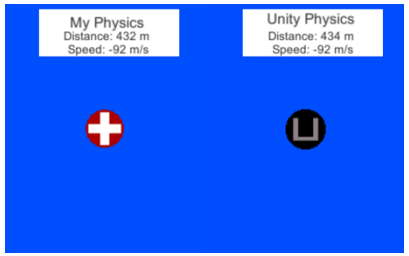
Future Development

e are almost limitless possibilities to add next, some of r developments I plan on making include:
nenting a closest pair of points algorithm in order to ize efficiency
n and bounciness variables
n on collision

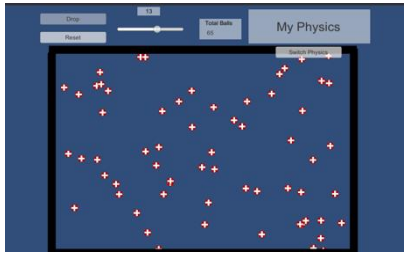
Testing



The first environment I created was a simple 5 ball max collision simulator. Users can enter different values that allow for changes in the environment and watch how Unity handles the collision versus how my physics engine handles the collision. My simulation is almost identical to Unity's with no significant drop in frame rate.



The second environment was a gravity simulation. Two balls drop one using Unity's engine one using my engine. Distinction in distance can be seen with a darkening blue color. By using the gravity constant 9.87m/s and a built-in variable Time.deltaTime to account for changes in frame rate the two balls fall in near perfect sync.



The final environment I set up was to allow as many objects as the user wants in order to maximize the amount of collisions until the frame rate dropped to essentially 0. Unity would start to drop frames and inaccurately calculate collisions at around 1000 objects. My engine was able to get to around 50 before doing the same.

Realtime video:

Design

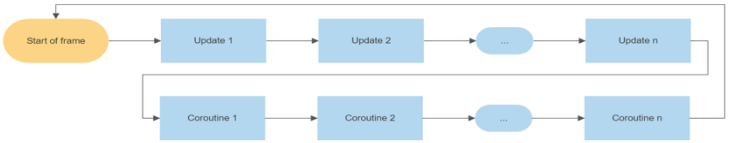
The design of my physics engine was like that of Unity's. Each object has both a rigid body and a collision script attached. The rigid body script handles the physics operations such as velocity and gravity, where the collision script handles collisions. Each component factors in it's own velocity, mass, and position as well as that of which it is interacting with to computes it's own values for the next frame.

Challenges

The main challenge was understanding and manipulating Unity's update system in order to ensure objects collide appropriately. Normally Unity's update runs linearly.

The issue with this is that if Object 1 collides with Object 2, Object 1 computes its velocity based on Objects 2 velocity and then updates its velocity. When Object 2's update is called it is now calculating it's new velocity on Object 1's already updated velocity. Hence, Object 2's new velocity is wrong.

To fix this problem I utilized Unity's built in Coroutine function. This allows for calculations to be made in the Update function and then waiting until all until all computations to be made to change values, resulting in correct collisions.



Before the Coroutines two objects would likely get stuck inside one another, but by using the Coroutine this problem no longer happens.

References

Unity Download: https://unity3d.com/get-unity/download
Gravity: https://gamedev.stackexchange.com/questions/15708/how-can-i-implement-gravity/41917#41917
Collisions: https://opentextbc.ca/physicstestbook2/chapter/collisions-of-point-masses-in-two-dimensions/
Project download: https://github.com/isaacBrinkman/PhysicsEngine